

CECS 381: Stochastic Computing

Programming Assignment 1 (Units 1–3: Simulation & Naive Bayes)

Summer 2026 Assigned: July 14 Due: July 23, 2026 (11:59 PM)

California State University, Long Beach

Name: _____ Student ID: _____

Instructions. Submit a single Jupyter notebook (.ipynb) plus a PDF export. Use only the Python standard library, `numpy`, and `matplotlib` (no `scikit-learn` for Problem 3). Your code must run top to bottom without errors. Comment your work and answer the written questions in markdown cells. Total: 100 points.

Question:	1	2	3	Total
Points:	30	30	40	100
Score:				

1. (30 points) **The Birthday Paradox.** Estimate, by simulation, the probability that in a room of n people at least two share a birthday (assume 365 equally likely days, no leap years).
 - (a) (12 pts) Write `simulate_birthday(n, trials)` that samples n birthdays per trial and returns the empirical probability of at least one shared birthday.
 - (b) (10 pts) Write `theoretical(n)` returning $1 - \prod_{k=0}^{n-1} \frac{365-k}{365}$. Print a table comparing simulation and theory for $n = 5, 10, \dots, 50$.
 - (c) (8 pts) Find (and print) the smallest n for which the theoretical probability exceeds 0.5. Plot both curves versus n .
2. (30 points) **Monte Carlo probability with dice.** Use random sampling to estimate probabilities and compare with theory.
 - (a) (15 pts) Roll two fair dice $N = 10^6$ times. Estimate the PMF of the sum (2 through 12) and compare $P(\text{sum} = 7)$ with the exact value $1/6$.
 - (b) (15 pts) Estimate the probability of rolling at least one six in four rolls of a single die, and compare with $1 - (5/6)^4$.
3. (40 points) **Naive Bayes spam classifier.** Using the `spam.csv` dataset from the Unit 2 lab (each row: a label `ham/spam` and a message), build a Naive Bayes classifier *from scratch*.
 - (a) (10 pts) Load the data, lowercase and tokenize each message into words, and split 80% train / 20% test (shuffle with a fixed seed).
 - (b) (20 pts) Implement multinomial Naive Bayes with Laplace (add-one) smoothing: estimate the class priors and per-class word likelihoods, and classify each test message with log-probabilities.
 - (c) (10 pts) Report accuracy, precision, and recall for the spam class. Briefly explain (markdown) why log-probabilities are used instead of multiplying raw probabilities.